

analysed offline on a development machine.

## Thread dumps

Thread dumps are generally used in identifying performance issues with slow running applications. They are also useful in identifying the causes of issues such as sporadic slowness and the unpredictable or greatly varying response times of applications. Thread dumps consist of information regarding all threads and the full stack trace of code running on each thread. They also give the state of each thread at the instant the dump was taken. Thread dumps taken from multiple runs of the same program can help in identifying race conditions in a program.

## Memory dumps

Memory dumps are useful in identifying memory issues with applications. These issues could be major ones such as memory leaks, which cause applications to crash over a period of time, or minor slowness issues arising out of excessive garbage collection. Depending on the JVM provider, there can be two kinds of dumps—heap dumps that contain only the contents of heap, and native memory dumps that consist of both heap and non-heap memory areas of the JVM. In this tutorial, only heap dumps will be explained as applications creating objects on a heap are the most common source of memory leaks. A heap dump consists of a list of all live objects on the heap, their sizes and the objects they contain.

## Memory dump formats

### HPROF format

Historically, HPROF was a profiling agent that was shipped along with the Sun JDK. When the JVM was run with the `-agentlib:hprof` option, it turned on the HPROF agent and collected profiling information that could be dumped to a file for analysis. This provided both memory and CPU profiling information. The default option for this HPROF agent was to dump the profiling information to a text file. The HPROF agent has been revamped since Java 5.0, and uses the JVM Tool Interface (JVMTI) as opposed to the JVM Profiling Interface (JVMPPI) used by the earlier HPROF agent. Advanced tools such as *jmap* (used with *jhat*) and *jvisualvm* now provide a better view of memory areas than the default text dump provided by the HPROF agent. Dumps from tools such as *jmap* and *jvisualvm* still follow the HPROF format. This binary format file can be used with the Memory Analysis Tool (MAT) or the Heap Analysis Tool (HAT) to examine objects on the heap. Currently, HPROF is the default dump format on Oracle JVMs.

### PHD format

Portable Heap Dump or PHD is a memory dump format

from IBM. This is the default format on IBM JVMs. Like HPROF, PHD can be either in text or binary formats. The PHD format is not supported on tools from Oracle such as *jhat*, *jvisualvm* and *JDeveloper*. A plugin for Eclipse MAT can be used to open and analyse PHD files.

## Oracle vs IBM dumps

Both thread and memory dumps can be triggered using command line utilities in Oracle JVMs. These are shipped along with the JDK. The thread dump from the *jstack* tool is in text format and the memory dump from the *jmap* tool is in HPROF format.

There are three kinds of dumps available on IBM JVM:

- 1) **Heap dump:** This dump is in the PHD format. It contains the heap information. The default extension for this dump is *.phd*
- 2) **System dump:** This also is a memory dump, but contains both heap and non-heap information about the Java process. The default extension for this file is *.dmp*
- 3) **Java Core dump:** This is a thread dump and is in text format. The default extension is *.txt*. Though it can be opened with any text editor, a thread dump analysis tool such as 'IBM Thread and Monitor Dump Analyzer for Java' presents the thread dump information in a readily consumable way, and also reports any deadlocks in the program.

## Modes of acquiring dumps

### Using an IDE

This method is mostly useful in development environments where the dumps can be taken from within the IDE such as Eclipse, Rational Application Developer (RAD), JDeveloper or Jvisualvm. Such IDE-based tools can attach to a running process and take a dump, but this feature may not be useful in a production environment, since it may not be possible to install the IDE on production machines. Though these tools can also be attached to a JVM port on a different machine, it still may not be possible to use it in production due to firewall restrictions and locked down server start-up scripts.

### Sending a signal to the JVM process

JVMs can be configured to dump memory when a crash occurs due to *OutOfMemoryError*. Additionally, most JVMs allow a signal to be sent to the JVM process to dump memory. The signal can be sent via a trigger such as *Ctrl+Break*. On UNIX machines, this can easily be done by the *kill* command: `kill -QUIT/kill -3 <process-id>`. On Windows machines, sending this signal could involve writing custom code that invokes a Windows native event such as *GenerateConsoleCtrlEvent()*. This method is the most preferred way to obtain a heap dump on IBM JVMs.